REI603M - Assignment 6

Progress, Monitoring & Next Steps

1. Overview

Work in the **same groups as before**. After the individual MAB competition (A5), you are returning to your group projects. The goal of this assignment is to make **substantial progress** on your application: extend your observability, improve your product, and plan your final sprint.

Pick **at least 3 items** from the Progress Menu below and make meaningful progress on each. You will present a 10-minute update with a demo on **March 11th**.

2. Submission Requirements

- **Deliverables:** Submit on Canvas:
  1. Presentation slides (**10 slides**, PDF)
  2. Link to your Git repository (shared with haffi112 on Github)
- **Deadline:** Tuesday March 10th, 2026 at 23:59 on Canvas
- **Presentations:** March 11th in class (**10 minutes** per group)

Important:

- The demo must be **live or a recorded screen capture**. No slide-only walkthroughs
- **No late submissions.** Late work will not be accepted
- You must pick **at least 3 items** from the Progress Menu

Suggested Team Roles (for groups of 2–4)

- **Monitoring & Evaluation Lead:** Observability, dashboards, alerting, evaluation pipeline
- **Feature & Prompt Lead:** New capabilities, prompt improvements, core product features
- **Infrastructure & DevOps Lead:** Deployment, CI/CD, containerization, environment management
- **UX & Data Lead:** UI/UX improvements, user interaction logging, analytics, data collection

For groups of 3, combine the Infrastructure & UX roles. For groups of 2, each person takes 2 roles.

3. Progress Menu

Pick **at least 3** of the following. For each item you choose, make meaningful progress and be prepared to show what you did in your presentation.

3.1. Extend Observability & Monitoring

Go beyond the basic Langfuse tracing from A4. Build dashboards to visualize cost, latency, and usage trends. Set up alerts for cost spikes or latency degradation. Track output quality over

time. Capture user feedback (thumbs up/down, corrections). Monitor token efficiency across different prompts or user flows.

References:
- Langfuse Analytics & Dashboards
- Langfuse Scores & Metrics
- Helicone – proxy-based LLM observability
- OpenTelemetry LLM Observability

3.2. Evaluation in Production

Run your evaluation dataset (from A3) against the live system. Compare results with your A3/A4 baselines to detect quality regression. Add LLM-as-judge scoring for dimensions like helpfulness, accuracy, or safety. Set up automated quality checks that run on a schedule or on every deployment.

References:
- DeepEval – open-source LLM evaluation framework
- Ragas – evaluation framework for RAG pipelines
- Langfuse Model-Based Evaluations

3.3. Guardrails & Safety

Add input/output validation to protect against misuse and failure modes. Implement content filtering, hallucination detection, PII handling, or structured output validation. Build graceful fallbacks for when the LLM produces invalid or unsafe outputs.

References:
- NVIDIA NeMo Guardrails
- Guardrails AI
- LLM Guard by Protect AI

3.4. Prompt Management & Experimentation

Implement prompt versioning so you can track which prompt version produced which outputs. Run A/B tests comparing prompt variants in production. Track prompt performance metrics over time. Document your prompt evolution from A3 through A4 to now.

References:
- Promptfoo – test and evaluate prompts
- Langfuse Prompt Management
- Braintrust

3.5. Try a New LLM Capability

Integrate something you haven't used yet: tool use / function calling, agentic workflows, MCP integration, multimodal inputs (images, audio), multi-model routing, or structured outputs. Pick something that adds real value to your application.

References:
- MCP Specification and MCP Python SDK
- Microsoft MCP for Beginners
- Anthropic's Building Effective Agents
- OpenAI Function Calling Guide

3.6. Deployment & Infrastructure

Deploy your application to the cloud so others can access it. Set up CI/CD for automated testing and deployment. Containerize with Docker. Add automated tests that run before deployment. Implement proper environment management for staging vs. production.

References:
- Docker Getting Started
- GitHub Actions Documentation
- Streamlit Cloud, Railway, Render – easy deployment options

3.7. UI/UX & User Experience

Improve your interface beyond the basic prototype. Add streaming responses so users see output as it's generated. Improve error messages and loading states. Consider accessibility and mobile responsiveness. Make the interaction feel polished.

References:
- Streamlit Documentation
- Gradio Documentation
- Chainlit Documentation
- Vercel AI SDK – for streaming UI patterns

3.8. Data Collection & Analytics

Set up systematic logging of user interactions: what users ask, how they use the product, where they get stuck. Build an analytics view or dashboard. Implement feedback collection mechanisms. Create a data pipeline that feeds insights back into product improvement.

References:
- Langfuse User Feedback
- PostHog – open-source product analytics

4. Feature Plan

List **at least 5 features or improvements** you plan to complete before the final submission. For each one, write one sentence explaining what it is and why it matters for your product.

This serves as your roadmap for the final sprint. You will revisit this list in the final assignment.

5. Presentation Structure

Your **10-slide** presentation should follow this structure. The **demo** should be 3–4 minutes of the 10-minute slot.

Slide	**Content**
1. Title + Team	Project name, team members, and roles
2. Quick Recap	What your product does (1 slide refresher)
3. Progress Overview	Which 3+ menu items you tackled, who did what
4. Monitoring & Observability	What you instrumented, dashboards, insights from your data
5. Technical Deep-Dive	The most interesting or challenging thing you built
6. Demo	Live demo or recorded screen capture (3–4 minutes)
7. What the Data Tells You	Metrics, evaluation results, quality trends
8. Lessons Learned	Engineering reflections, what surprised you, what you would do differently
9. Next 5 Features	What you plan to build before the final submission
10. Architecture Diagram	Current system state: components, data flow, integrations

6. Grading

Your grade is based on **submitting your work and presenting according to the instructions**.

Deductions: −0.5 for each missing element

Examples of "missing elements":

Progress Menu:
- Fewer than 3 items attempted from the Progress Menu
- No evidence of meaningful work on a claimed item
- No demo or demo does not show working functionality

Monitoring & Observability:
- No monitoring beyond what was done in A4
- No dashboard, metrics view, or data insights shown
- No discussion of what the observability data revealed

Feature Plan:
- Fewer than 5 planned features listed

- Features lack explanation of why they matter

Presentation:
- Missing a slide from the prescribed structure
- No live demo or recorded screen capture
- Presentation exceeds 10 minutes significantly
- Slides not submitted by deadline
- No link to Git repository provided

For any questions, contact: `hafsteinne@hi.is`.

7. Resources

Observability & Monitoring

- Langfuse: Open-source LLM observability with tracing, scoring, and prompt management
- Helicone: Proxy-based LLM observability, minimal code changes
- PostHog: Open-source product analytics
- OpenTelemetry LLM Observability

Evaluation & Safety

- DeepEval: Open-source LLM evaluation framework
- Ragas: Evaluation framework for RAG pipelines
- Promptfoo: Test and evaluate LLM prompts
- NeMo Guardrails: Programmable guardrails for LLM applications
- Guardrails AI: Input/output validation for LLM apps
- LLM Guard: Security toolkit for LLM interactions

New Capabilities & Patterns

- Building Effective Agents: Anthropic's guide to composable agent patterns
- Model Context Protocol (MCP): Open standard for connecting LLMs to tools and data
- MCP for Beginners: Microsoft's introductory guide to MCP
- OpenAI Function Calling Guide

Deployment & Infrastructure

- Docker Getting Started
- GitHub Actions: CI/CD automation
- Streamlit Cloud, Railway, Render: Easy deployment platforms